

Software Development Plan

Autonomous Delivery Cart System

Project Title:	Autonomous Delivery Cart System
Version:	1.0
Date:	1/10/2025
Authors:	
Supervisors:	

Contents

1. Introduction.....	2
2. Project Organization	2
3. Project Timeline & Sprints.....	3
4. Project Deliverables	7
5. Risk Management.....	7
6. References.....	8

1. Introduction

1.1 Purpose

This document outlines the comprehensive development plan for the Autonomous Delivery Cart System; a graduation project aimed at creating a proof-of-concept for an automated last-mile delivery solution. It details the project scope, timeline, team responsibilities, development methodology, and key deliverables required to successfully build and deploy the system.

1.2 Project Scope

The project encompasses the design, development, and integration of four primary components:

1. **A cloud-native backend** hosted on Amazon Web Services (AWS).
2. **A cross-platform customer mobile application** for ordering and tracking.
3. **A web-based portal** for vendors to manage orders and dispatch carts.
4. **On-board software** for the physical delivery cart to communicate with the cloud.

The goal is to deliver a Minimum Viable Product (MVP) demonstrating a full, end-to-end delivery cycle.

2. Project Organization

2.1 Team Roles & Responsibilities

This project will be executed by a team of two, with clear areas of ownership to ensure parallel development and efficiency.

- **Ahmed (Cloud Architect & Software Developer):**
 - Responsible for the design and implementation of the entire AWS cloud infrastructure using Infrastructure as Code (IaC).
 - Develops all backend business logic using AWS Lambda.
 - Builds and deploys the Vendor and Manager web applications.
 - Manages the CI/CD pipeline and overall system integration.
- **Youssef (Mobile Application Developer):**
 - Responsible for the design, development, and testing of the customer-facing mobile application (iOS and Android).
 - Implements the user interface (UI) and user experience (UX).
 - Integrates the mobile application with the AWS backend via API Gateway and AWS Amplify.

2.2 Tools & Technologies

Category	Technology
Version Control	Git, GitHub
Cloud Provider	Amazon Web Services (AWS)
Infrastructure as Code	AWS CDK (Python)
Backend Logic	AWS Lambda (Node.js or Python)
Databases	Amazon RDS (PostgreSQL), Amazon DynamoDB
API Layer	Amazon API Gateway (REST)
Authentication	Amazon Cognito
Real-Time Communication	AWS IoT Core (MQTT)
Mobile App	Flutter or React Native
Web Apps	Python
Project Management	Trello, Jira, or GitHub Projects

2.2 Development Methodology

The project follows an **Agile methodology** using **two-week sprints**. Each sprint concludes with a functional review and retrospective.

Continuous integration and deployment (CI/CD) will be applied using AWS CodePipeline and GitHub Actions to ensure smooth and consistent deployments.

Daily communication between Ahmed and Youssef will be maintained through Slack or WhatsApp, with weekly status meetings held with the supervisor.

3. Project Timeline & Sprints

The project is structured into **three** major phases, broken down into eight two-week sprints. This agile approach allows for iterative development, regular integration, and the flexibility to adapt to challenges.

Phase 1: Foundation & Authentication (Sprints 1-2)

- **Goal:** Establish the development environment and build the core user login/registration feature.

Sprint	Goal & Key Deliverables	Tasks & Responsibilities
Sprint 1 (2 Weeks)	Setup & "Hello, Cloud" - GitHub repository is created and configured. - AWS & mobile environments are set up. - Initial IaC deployment is successful.	Ahmed: Create Git repo, set up AWS IAM user, initialize CDK project, deploy a test resource (S3 bucket). Youssef: Clone repo, set up mobile dev environment, create placeholder UI screens and navigation.
Sprint 2 (2 Weeks)	User Authentication Loop - Cognito User Pool is live. - Users can register and log in via the mobile app. - App has protected routes for logged-in users.	Ahmed: Define and deploy Cognito User Pool via CDK, provide UserPoolId/ClientId to Youssef. Youssef: Integrate AWS Amplify, connect UI to Cognito functions, implement auth-based navigation.

Phase 2: Core MVP Functionality (Sprints 3-5)

- **Goal:** Implement the main feature: a customer can place an order, a vendor can see it, and the cart is connected.

Sprint	Goal & Key Deliverables	Tasks & Responsibilities
Sprint 3 (2 Weeks)	Vendor & Order APIs - Database tables for Orders/Vendors are created. - Secured API endpoints for creating and viewing orders are live. - Mobile app can successfully place an order.	Ahmed: Define DB tables and Lambda functions for orders via CDK. Create secured API Gateway endpoints. Youssef: Build UI for browsing items and placing an order. Connect UI to the new POST /orders API.
Sprint 4 (2 Weeks)	Vendor Dashboard MVP - A basic web portal for vendors is deployed. - Vendors can log in and view a list of new orders. - CI/CD for the web app is functional.	Ahmed: Initialize web project for vendor portal, implement Cognito login, deploy using Amplify Hosting or S3/CloudFront. Team: Connect the dashboard to the GET /orders API to display data.
Sprint 5 (2 Weeks)	Basic Cart Integration - AWS IoT Thing for the cart is configured. - Cart can securely connect to AWS IoT Core. - Cart telemetry is being received and stored in DynamoDB.	Ahmed: Use CDK to provision IoT resources. Write the ingestion Lambda and IoT Rule. Team: Write and test the Python script on the Jetson to connect and publish data.

Phase 3: Real-Time Features & Analytics (Sprints 6-8)

- **Goal:** Bring the system to life with live tracking, a manager dashboard, and remote cart control.

Sprint	Goal & Key Deliverables	Tasks & Responsibilities
Sprint 6 (2 Weeks)	Real-Time Tracking & Notifications - API endpoint for fetching cart location is live. - Mobile app displays the cart's location on a map. - Users receive an notification on order placement.	Ahmed: Create the tracking API and Lambda. Integrate Amazon SNS for notifications. Youssef: Build the map view UI. Implement logic to periodically call the tracking API and update the map marker.
Sprint 7 (2 Weeks)	Manager Dashboard & Analytics - Amazon QuickSight is set up with data sources. - A basic dashboard showing key metrics is created. - The dashboard is embedded in a secure web portal for managers.	Ahmed: Set up QuickSight, create analyses (e.g., orders per day). Build the manager web portal and embed the dashboard. Implement Cognito login.
Sprint 8 (2 Weeks)	Cart Command & Control - API for the "unlock" command is functional. - The backend can send an unlock command to the cart via MQTT. - The cart's software can receive and act on commands.	Ahmed: Create the POST /unlock API and Lambda. The Lambda will publish a command to an IoT topic. Team: Update the Jetson script to subscribe to the command topic and simulate an unlock action.

4. Project Deliverables

Phase 1: AWS infrastructure setup, Cognito authentication, mobile app login.

Phase 2: Working order workflow, vendor dashboard MVP, cart connection.

Phase 3: Real-time tracking, analytics dashboard, cart control simulation.

Final Submission: Complete technical documentation (SRS, SDD, SDP), functional prototype demo, and architecture diagrams.

5. Risk Management

Potential Risk	Likelihood	Impact	Mitigation Strategy
Hardware Integration Delays	Medium	High	Begin basic cart connection (Sprint 5) early. Simulate hardware behaviors in software to allow parallel development.
AWS Budget Overruns	Low	Medium	Strictly adhere to the AWS Free Tier. Set up billing alerts to monitor costs proactively.
API Integration Conflicts	Medium	Medium	Maintain clear API documentation (e.g., using Postman or OpenAPI). Hold regular (daily/bi-daily) sync-up meetings between Ahmed and Youssef.
Scope Creep	Medium	High	Strictly adhere to the features

			defined in the SRS and this plan. Any new feature requests must be formally approved by supervisors.
--	--	--	--

6. References

AWS Well-Architected Framework (Amazon Web Services, 2024)
IEEE Std 1058-1998 – Software Project Management Plans